

Today's Agenda:-

Starting 7:05

1. Space Complexity
2. Intro to Arrays
3. Reverse an Array
4. Rotate an Array
5. Dynamic Arrays
6. TLE error
7. Importance of Constraints

Revision Quiz

$$4N^2 + 3N + 1$$

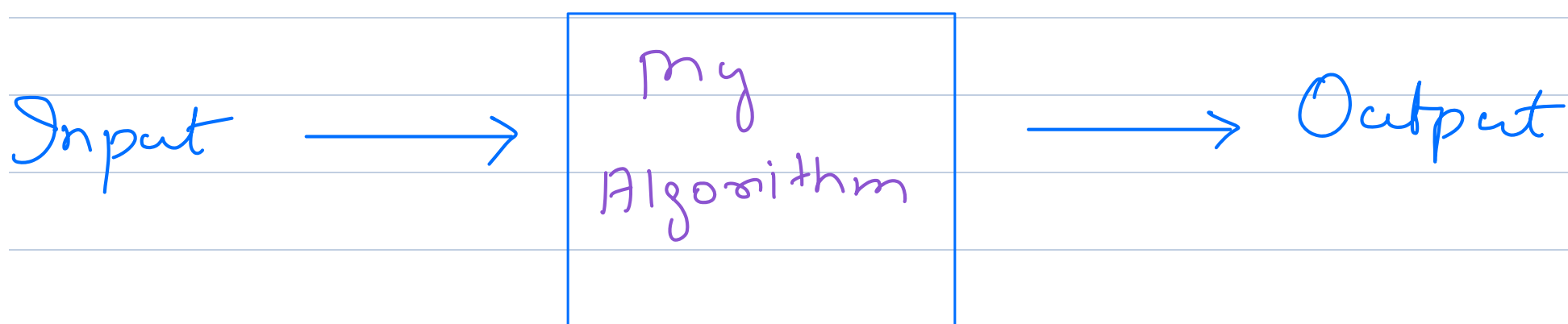
$$= \underline{\underline{O(N^2)}}.$$

Space Complexity:

space point a Maximum auxiliary / extra that is utilised at any during the execution of program.

→ Also represented using Big O

→ Calculated similarly like T.C.



Any space utilised by my algo will contribute in the SC.

Example:-

```
void func (int N) {  
    int x; // 4 B  
    int y; // 4 B  
    long z; // 8 B  
}
```

16 B ; SC $\rightarrow$ $O(1)$.

Q-3.

Find the Space Complexity [Big(O)] of the below program.

```
func(int N) {  
    int arr[10]; // 40 Bytes  
    int x; // 4 bytes  
    int y; // 4 bytes  
    long z; // 8 bytes  
    int arr[N]; // 4 * N bytes  
}
```

$(4N + 56)$ Bytes.

SC $\rightarrow$ $O(N)$.

Q-4.

Find the Space Complexity [Big(O)] of the below program.

```
func(int N) {           // 4 bytes
    int x = N;           // 4 bytes
    int y = x * x;       // 4 bytes
    long z = x + y;      // 8 bytes
    int arr[N];           // 4 * N bytes
    long l[N][N];        // 8 * N * N bytes
}
```

$$8N^2 + 4N + 16$$

$$S.C \rightarrow O(\underline{N^2})$$

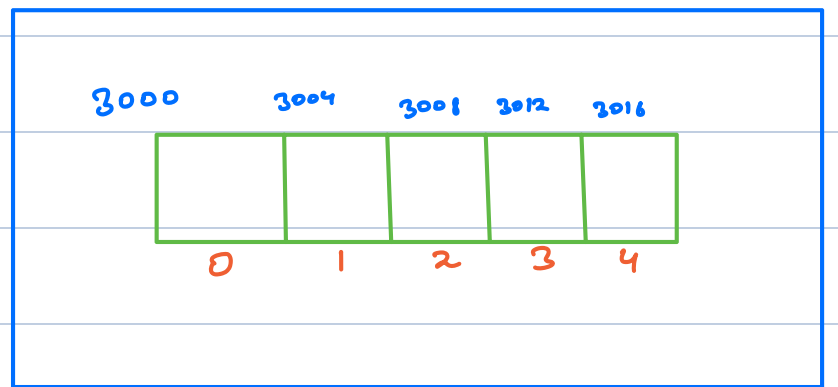
Introduction To Arrays:

A collection of elements of same data type stored in continuous memory locations.

ex:- int ; double ; float ; char ;

int
↑
datatype

arr[5];
↑
reference of the array.
↓
No. of elements.



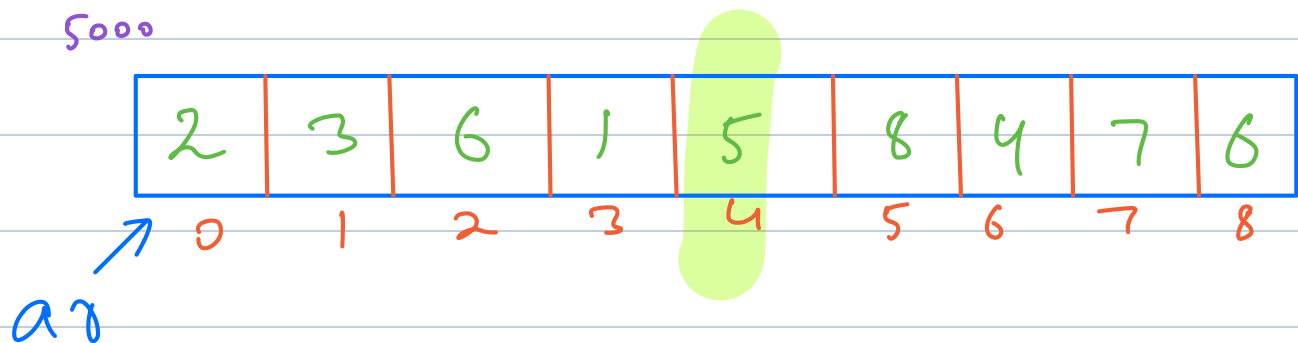
Heap.

ex:-

arr[2];

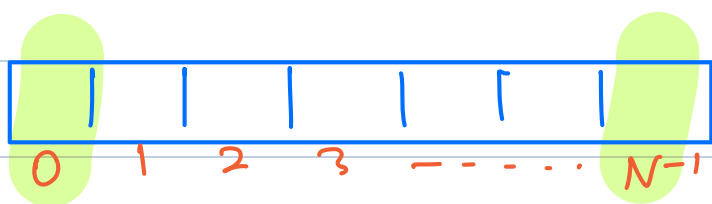
36 B

int arr[9] = { 2, 3, 6, 1, 5, 8, 4, 7, 6 }



5th element → print (arr[5-1])
↓
arr[4];

Q:- 5



index 0 $\rightarrow$ 1st
index N-1 $\rightarrow$ Nth element.

Q:- 6

print (arr[i]) $\rightarrow$ O(1).

arr[i] $\rightarrow$ * (arr + i * (Size of each element)) $\rightarrow$ 2.

$\rightarrow$ * (3000 + 2 * (4))
 $=$ * (3008).

arr[i] = 5;

Q:- 7

int arr[5] = {⁰5, ¹-4, ²8, ³9, ⁴10}

print (arr[0] + arr[4])

$\rightarrow$ O(1).

Printing an array

$[0, N-1]$

for ($i=0$; $i \leq N-1$; $i++$) {

$\downarrow$
 $N-0$ $\neq$

| print (arr[i]);

= $\textcircled{N}$

3

$T.C \rightarrow \underline{O(N)}$

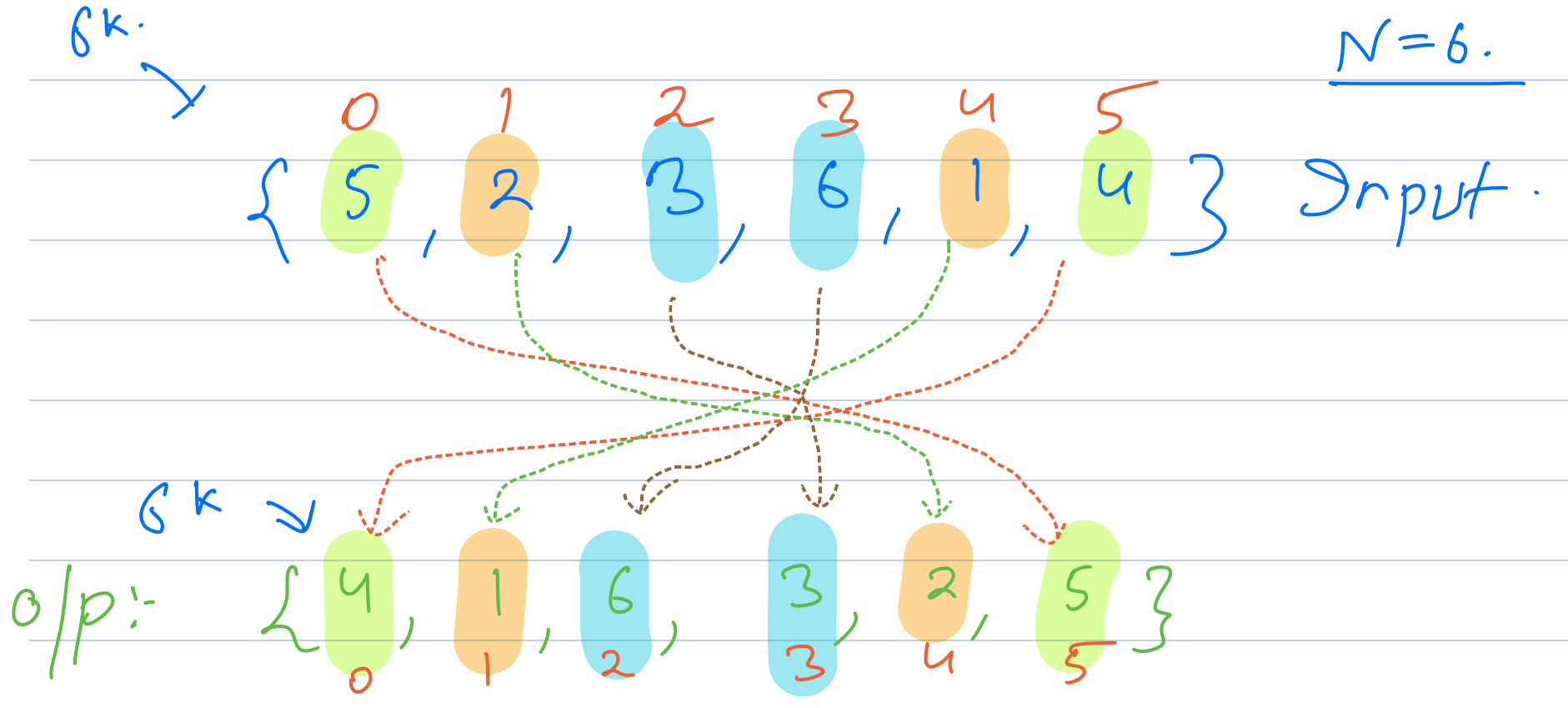
Q:- Reverse the array

$N=5$

arr = { 1, 2, 3, 4, 5 } (in-place)

$\downarrow$

o/p:- 5, 4, 3, 2, 1

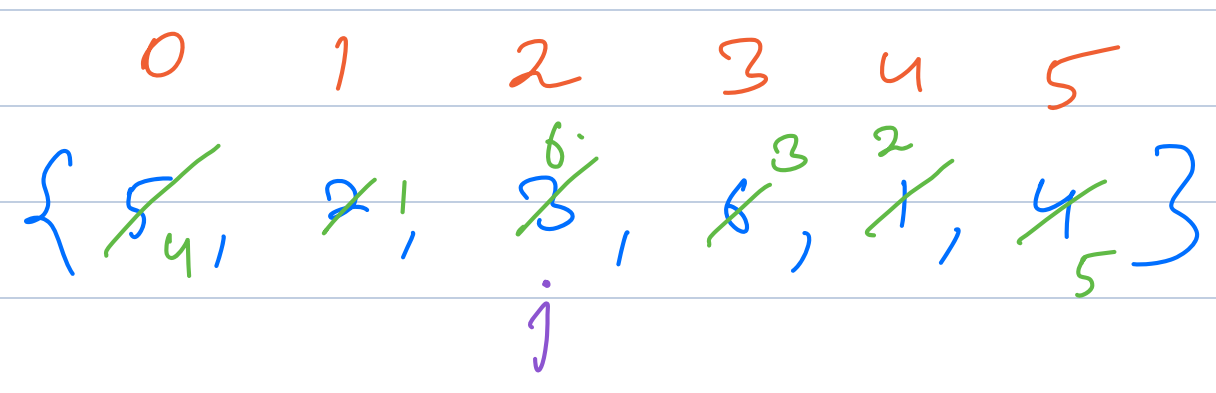


Observation:-

After reversal, the first & the last elements have swapped their positions.

Same " for 2nd & 2nd last, 3rd & 3rd last & to on.

Approach :-



```
temp = arr[i]
arr[i] = arr[j];
arr[j] = temp;
```

Take two variables i & j & make them point at the start & ending of the array.

→ Swap the values

→ $i++$; $j--$; Continue until $i \leq j$


```
int[] solve (int arr[], int N) {
```

```
    return reverse (arr, N, 0, N-1);
```

```
}
```

```
int[] reverse (int arr[], int N, int s, int e) {
```

```
    int i = s; int j = e; int temp;
```

```
    while (i < j) {
```

```
        temp = arr[i];
```

```
        arr[i] = arr[j];
```

```
        arr[j] = temp;
```

} Swapping

```
        i++; j--;
```

```
    }
```

```
    return arr;
```

```
}
```

$$K_1 + K_2 \cdot \frac{N}{2}$$

T.C $\rightarrow O(N)$

12B ;

S.C $\rightarrow O(1)$

Reversing an array in a range

$\text{arr}[5] = \{ 1, 2, 3, 4, 5 \}$

Indices: 0 1 2 3 4
Start: $s = 1$
End: $e = 3$

$s = 1$

$e = 3$

↓
o/p:- $\{ 1, 4, 3, 2, 5 \}$

```
int[] solve (int[] arr, int N, int s, int e) {
```

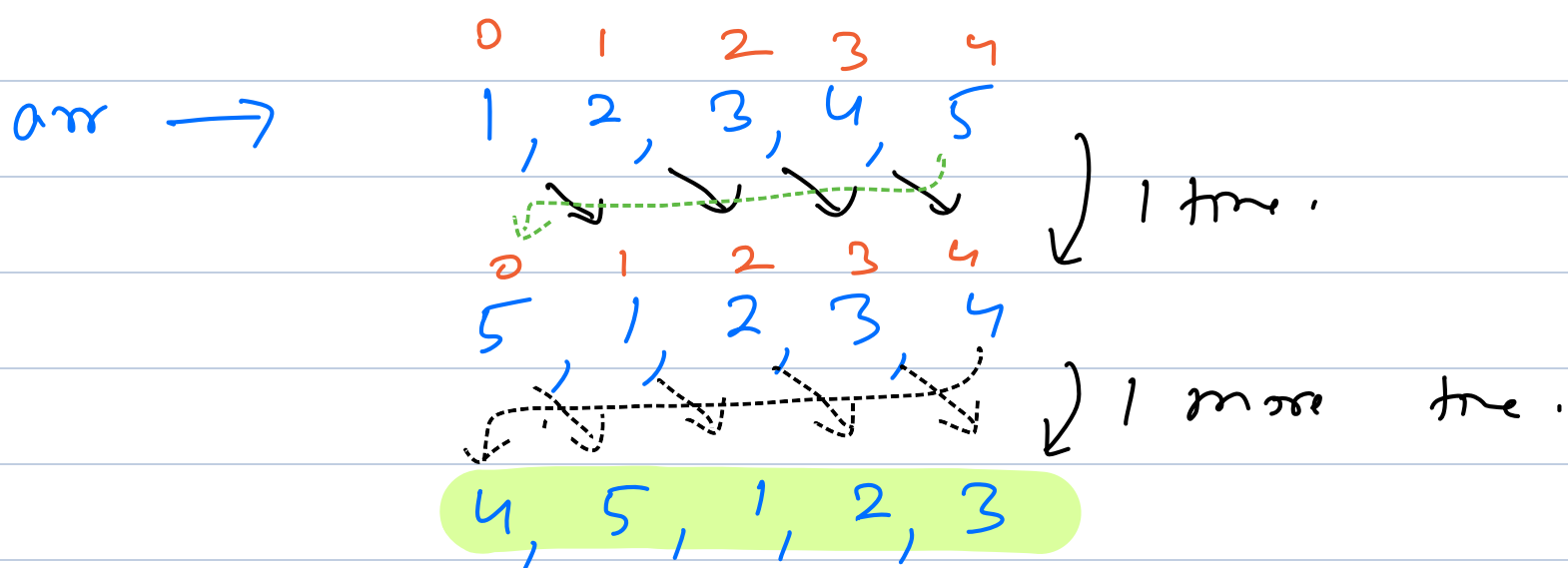
```
    return reverse (arr, N, s, e);
```

}

Q:- Rotate array k times

Given an array of size N and an integer k. Rotate the array by k positions clockwise.

ex:- $arr = \{1, 2, 3, 4, 5\}$
 $k = 2$



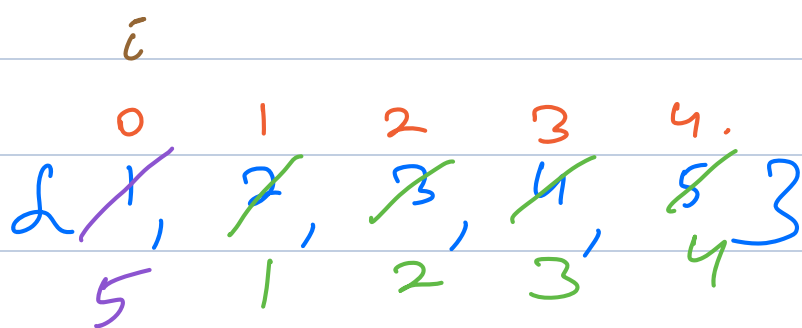
Brute Force

Rotate the array by 1 in a C.W. direction k times.

Array Index Out of Bound.

$arr[-1]$

Simple Rotation



$0 \rightarrow N-1$

$int\ temp = arr[N-1];$

$arr[i] = arr[i-1]$

$arr[0] = temp;$

Code.

```
int[] rotate (int arr[], int N, int k) {
```

```
    int i, temp, rot; k1.
```

```
    for (rot = 1; rot <= k; rot++) {
```

```
        temp = arr[N-1]; k2
```

```
        for (i = N-1; i > 1; i--) {
```

```
            arr[i] = arr[i-1]; k3
```

```
        arr[0] = temp; k4
```

```
    }
    return arr;
```

S.C $\rightarrow$ $O(1)$.

$$k_1 + k(k_4 + k_2 + k_3(N-1))$$

$$k_1 + k \cdot k_4 + k \cdot k_2 + k_3 N \cdot k - k_3 \cdot k$$

$$k_1 + k(k_2 + k_4 - k_3) + N \cdot k \cdot k_3$$

T.C $\rightarrow$ $O(N \cdot k)$.

Q:- Can we optimise this T.C.

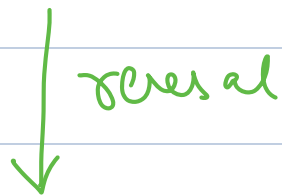
arr: { ^{0 1 2 3 4 5 6}
1, 2, 3, 4, 5, 6, 7 } ; k=3



o/p: { 5, 6, 7, 1, 2, 3, 4 }

(1) Reverse the array.

{ 1, 2, 3, 4, 5, 6, 7 }



{ 7, 6, 5, 4, 3, 2, 1 }
_{0 1 2 3 4 5 6}

(2) Reverse first k elements & reverse remaining N-k elements

{ 7, 6, 5, 4, 3, 2, 1 }
_{0 1 2 3 4 5 6}



{ 5, 6, 7, 1, 2, 3, 4 }

Code.

int[] Solve (int[] arr, int N, int k) {

$k = k \% N$ (Rotations repeat after N).

reverse (arr, N, 0, N-1);

reverse (arr, N, 0, k-1);

reverse (arr, N, k, N-1);

}

T.C $\rightarrow O(N)$.

S.C $\rightarrow O(1)$.

8:42.

Dynamic Arrays

Fast Access.

Fixed Sze. $\rightarrow$ Dynamic Arrays
address this limitation
of arrays.

Java :- ArrayList

C++ :- vector

```
vector<int> v;  
v.push_back(5);
```

Python :- List.

Retrieving $\rightarrow \underline{\Theta(1)}$.

Addn / Insertn $\rightarrow$ Amortised $\Theta(1)$
 $\downarrow$
Average.

Time Limit Exceeded (T.L.E.) ✖✖

Consider a student **Mehta** who likes **Amazon** and he is currently in a **hiring contest** organized by Amazon.

- He is given **two questions** and **1 hour** duration.
- He solves the **first** question, submits the code and finds it to be **TLE**.
- He again **changes the idea**, modifies the code and submits it, but he again gets **TLE**.

By the third time, the contest is already over.

- Is it necessary to write the entire code and then test it to determine its correctness? No.
- Can we assess the logic's viability before writing any code? Yes.

Online Editors & Why TLE occurs ?

→ Code is executed on online servers of various platforms like **Codechef**, **Codeforces** etc.

→ The processing speed of their machine is **1 GHz** which means they can perform **10^9** instructions per second.

→ Generally, the code should be executed within **1 sec** of time.

At max **10^9** instructions.

Code

$a++;$

$\boxed{5}_a$

$a = a + 1$

:

```
bool countFactors ( int N ) {  
    int c = 0;  
    for ( i = 1 ; i ≤ N ; i++ ) {  
        if ( N % i == 0 ) {  
            c = c + 1;  
        }  
    }  
}
```

// Res.

In my code, every iteration
takes 10 - 100 instructions.

Case I: 1 iteration $\rightarrow$ 100 instructions.

10^9 ins. $\rightarrow$ 1 sec.

100 ins $\times 10^7 \rightarrow$ 1 sec.

1×10^7 iteration $\rightarrow$ 1 sec.

Con II :- 1 iteration $\rightarrow$ 10 instructions

10×10^8 insts. $\rightarrow$ 1 sec

10^8 iterations $\rightarrow$ 1

$\therefore$ At max iterations my code can perform is b not give a TLE in b/w

$10^7 - 10^8$

Approaching a Problem

$$\begin{aligned} 1 &\leq N \leq 10^5 \\ 1 &\leq a[i] \leq 10^9 \end{aligned}$$

V.V. Imp

- Read the **Question** and **Constraints** carefully.
- Formulate an **Idea** or **Logic**.
- Verify the **Correctness** of the Logic.
- Mentally develop a **Pseudocode** or rough **Idea of Loops**.
- Determine the **Time Complexity** based on the Pseudocode.
- Assess if the time complexity is feasible and won't result in **Time Limit Exceeded (TLE)** errors.
- **Re-evaluate** the **Idea/Logic** if the time constraints are not met; otherwise, proceed.
- **Code** the idea if it is deemed feasible.

Importance of Constraints

$$1 \leq N \leq 10^5$$

Complexity	Iterations	Works?
$O(N^3)$	$(10^5)^3 = 10^{15}$	<u>Never</u> !
$O(N^2)$	$(10^5)^2 = 10^{10}$	<u>Never</u> !
$O(N \log N)$	$10^5 \times \log 10^5$ $\approx 10^7$	<u>Yes</u> !

$$1 \leq N \leq 10^6$$

Complexity	Iterations	Works?
$\Theta(N^2)$	10^{12}	<u>No</u> ;
$\Theta(N \log N)$	$10^6 \approx \log 10^6$ $\approx \underline{10^8}$	It may work. <u>Toy</u> .
$\Theta(N)$	$\underline{10^6}$	<u>100% work.</u>

$$1 \leq N \leq 100$$

$$\underline{\underline{\Theta(N^3)}}$$

100%

$$1 \leq N \leq 20$$

Even $\Theta(2^N)$ will work.

$$2^{20} = 2^{10} \times 2^{10}$$

$$= 1024 \times 1024$$

$$\underline{\underline{(10^7 - 10^8)}}$$

$$\approx \underline{10^8}$$

2, 4, 8, 16, 32, 64, 128, 256, 512, 1024
1 2 3 4 5 6 7 8 9 10